

Étude du Chiffrement Totalement Homomorphe : De DGHV à TFHE et CKKS

Constantin Delahodde, Martin Chedozeau, Lucas Schaeffer

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
1.1	Contexte historique et évolution	2
1.1.1	Une quête de trente ans : des prémices à la rupture de Gentry . . .	2
1.1.2	De la théorie à l'efficacité : vers une cryptographie multi-facettes . .	2
1.2	Outils théoriques et prérequis	3
1.2.1	Circuits booléens et portes logiques	3
1.2.2	Lien avec le chiffrement homomorphe	4
2	Le schéma DGHV : Chiffrement sur les entiers	5
2.1	Le problème AGCD	5
2.2	Chiffrement Symétrique ("Somewhat Homomorphic")	5
2.3	Chiffrement Asymétrique	6
2.4	Propriétés homomorphes et dynamique du bruit	6
2.5	Le Bootstrapping	7
3	TFHE : Chiffrement sur le Tore	8
4	CKKS : Calcul sur les nombres réels	9

Chapitre 1

Introduction

1.1 Contexte historique et évolution

La confidentialité des données numériques repose traditionnellement sur un compromis binaire : la donnée est soit sécurisée (chiffrée et illisible), soit utile (en clair et traitable). Ce paradigme impose que toute manipulation de données sensibles nécessite une phase de déchiffrement, créant ainsi une faille de sécurité critique au moment même où l'information est traitée en mémoire vive. Le **Chiffrement Totalement Homomorphe** (FHE, pour *Fully Homomorphic Encryption*) a été conçu pour briser cette dualité en permettant l'exécution d'algorithmes arbitraires directement sur des cryptogrammes.

1.1.1 Une quête de trente ans : des prémices à la rupture de Gentry

L'histoire du FHE débute en 1978, un an seulement après l'invention du RSA. Ronald Rivest, Leonard Adleman et Michael Dertouzos introduisent le concept de « *privacy homomorphisms* ». Ils pressentent déjà l'immense potentiel d'un système où une opération sur les chiffrés correspondrait à une opération sur les clairs. Si certains schémas précoces présentaient des propriétés homomorphes partielles — comme le RSA (homomorphe pour la multiplication) ou le cryptosystème de Paillier en 1999 (homomorphe pour l'addition) — la capacité à combiner **simultanément** additions et multiplications est restée le « Saint Graal » de la cryptographie pendant trois décennies.

Le verrou technologique saute en 2009 lorsque Craig Gentry, dans sa thèse de doctorat à l'Université de Stanford, publie le premier schéma totalement homomorphe. Sa percée repose sur une idée révolutionnaire : le *bootstrapping*. Puisque chaque opération homomorphe augmente le bruit inhérent au chiffré jusqu'à rendre le déchiffrement impossible, Gentry propose de déchiffrer le texte chiffré *sous sa forme chiffrée* pour réinitialiser ce bruit. Cette preuve d'existence a transformé un rêve théorique en une réalité mathématique, bien que les premiers temps d'exécution se comptaient alors en jours pour une simple opération.

1.1.2 De la théorie à l'efficience : vers une cryptographie multifacettes

À la suite des travaux de Gentry, la recherche s'est orientée vers la réduction de la complexité algorithmique et la diversification des structures mathématiques sous-jacentes.

On distingue alors plusieurs générations de schémas, passant des réseaux idéaux à des problèmes plus stables comme le *Learning With Errors* (LWE). Ce mémoire se propose d’analyser cette évolution à travers trois protocoles emblématiques qui illustrent la maturité actuelle du domaine :

- **Le schéma DGHV (2010)** : Proposé par van Dijk, Gentry, Halevi et Vaikuntanathan, il simplifie la construction de Gentry en remplaçant les réseaux complexes par l’arithmétique des entiers. L’étude de DGHV est pédagogiquement essentielle pour comprendre la gestion du bruit via le problème du diviseur commun approximatif (AGCD).
- **Le schéma TFHE (2016)** : Représentant de la troisième génération, le *Torus FHE* se distingue par son optimisation radicale du *bootstrapping*. En travaillant sur le tore réel, il permet d’évaluer des portes logiques (NAND, XOR) en quelques millisecondes, ouvrant la voie à l’exécution de circuits booléens complexes et de microprocesseurs chiffrés.
- **Le schéma CKKS (2017)** : Conçu par Cheon, Kim, Kim et Song, ce schéma marque un tournant pragmatique. Contrairement aux schémas exacts, CKKS traite le bruit comme une erreur d’arrondi inhérente au calcul en virgule flottante. Cette approche « approximative » le rend incomparablement plus efficace pour le *Machine Learning* et le traitement de données statistiques à grande échelle.

L’enjeu de ce travail sera de démontrer comment ces constructions mathématiques, bien que radicalement différentes dans leurs approches, convergent vers un objectif commun : rendre le traitement de données privées aussi fluide et universel que le calcul en clair.

1.2 Outils théoriques et prérequis

Pour analyser les mécanismes du chiffrement homomorphe, il est nécessaire de définir le modèle de calcul sous-jacent. Les ordinateurs ne manipulent pas des fonctions mathématiques complexes de manière native, mais évaluent des réseaux de portes logiques traitant des données binaires.

1.2.1 Circuits booléens et portes logiques

En informatique théorique, un algorithme peut être modélisé sous la forme d’un circuit booléen, c’est-à-dire un graphe orienté acyclique composé de portes logiques élémentaires.

Définition 1.1 (Portes logiques sur $\mathbb{F}_2$). Une porte logique est une fonction mathématique opérant sur un ou plusieurs bits (0 ou 1). Les deux opérations fondamentales pour le chiffrement homomorphe correspondent à l’arithmétique dans le corps fini $\mathbb{F}_2$:

- **La porte XOR (OU exclusif)** : Elle correspond strictement à l’addition modulo 2. Elle renvoie 1 si et seulement si un seul des deux bits d’entrée vaut 1.
- **La porte AND (ET)** : Elle correspond strictement à la multiplication modulo 2. Elle renvoie 1 si et seulement si les deux bits d’entrée valent 1.

L’intérêt exclusif pour ces deux portes logiques dans l’étude des schémas de chiffrement repose sur un théorème fondamental de l’informatique théorique.

Proposition 1.1 (Universalité du sous-ensemble {AND, XOR}). *L’ensemble de portes logiques {AND, XOR} est Turing-complet, sous réserve de disposer de la constante 1*

(permettant de simuler l'inversion logique *NOT* via $x \text{ XOR } 1$). Cela implique que toute fonction calculable, quelle que soit sa complexité, peut être traduite en un circuit combinatoire composé uniquement d'additions et de multiplications modulo 2.

1.2.2 Lien avec le chiffrement homomorphe

Cette universalité définit le critère de succès d'un schéma *Fully Homomorphic Encryption*. Si un protocole cryptographique parvient à simuler l'addition de deux textes chiffrés (équivalent à une porte XOR sur les clairs) et la multiplication de deux textes chiffrés (équivalent à une porte AND sur les clairs) sans destruction de l'information par le bruit, il devient alors théoriquement capable d'évaluer n'importe quel programme informatique à l'aveugle. L'enjeu des schémas DGHV, TFHE et CKKS réside dans les mécanismes mathématiques déployés pour atteindre cette capacité d'évaluation infinie.

Chapitre 2

Le schéma DGHV : Chiffrement sur les entiers

2.1 Le problème AGCD

La sécurité du schéma DGHV repose sur le problème du Plus Grand Commun Diviseur Approximatif (AGCD).

Problème 2.1 (AGCD). Soit un entier secret p . Si l'on fournit plusieurs multiples exacts $x_i = p \cdot q_i$, un simple calcul de PGCD permet de retrouver p . Cependant, si l'on ajoute une petite perturbation ou "bruit" r_i à ces multiples ($x_i = p \cdot q_i + r_i$), retrouver p devient extrêmement complexe. C'est ce principe qui masque la clé dans le DGHV.

Exemple 2.1 (Vulnérabilité sans bruit vs Protection). Le bruit n'est pas un défaut, mais un composant structurel indispensable. Supposons un scénario sans bruit où un attaquant intercepte deux chiffrés : $x_1 = 91$ et $x_2 = 143$. En appliquant l'algorithme d'Euclide, il calcule $\gcd(91, 143) = 13$. Il retrouve immédiatement la clé secrète $p = 13$.

En introduisant une perturbation, par exemple $x_1 = (13 \times 7) + 2 = 93$ et $x_2 = (13 \times 11) + 3 = 146$, l'attaque échoue car $\gcd(93, 146) = 1$. Le bruit casse la structure algébrique directe et assure l'indistinguabilité des textes chiffrés.

2.2 Chiffrement Symétrique ("Somewhat Homomorphic")

Définition 2.1 (Paramètres). Soit λ le paramètre de sécurité.

- p : clé secrète, entier impair d'une taille de λ^2 bits.
- $b \in \{0, 1\}$: le bit de message.
- q : grand entier aléatoire ($\sim \lambda^5$ bits).
- r : petit entier aléatoire représentant le bruit ($\sim \lambda$ bits).

Proposition 2.1 (Chiffrement et Déchiffrement). *Le chiffrement de b produit le texte chiffré c suivant :*

$$c = q \cdot p + 2r + b$$

L'extraction s'effectue avec la clé p par :

$$b = (c \bmod p) \bmod 2$$

Démonstration. Puisque p est beaucoup plus grand que $2r + b$, l'opération $c \bmod p$ supprime le multiple $q \cdot p$ et renvoie exactement $2r + b$. Ensuite, comme $b \in \{0, 1\}$ et que $2r$ est un nombre pair, $(2r + b) \bmod 2 = b$. $\square$

2.3 Chiffrement Asymétrique

Définition 2.2 (Génération des clés). — **Clé secrète (sk)** : L'entier impair p .

— **Clé publique (pk)** : Une suite de $\tau + 1$ entiers $(x_0, x_1, \dots, x_\tau)$. Chaque x_i est un "presque multiple" public de p , généré via la formule $x_i = q_i \cdot p + 2r_i$.

La liste est réorganisée pour que x_0 soit le plus grand entier. De plus, x_0 doit être impair, et $x_0 \bmod p$ doit être un nombre pair.

Proposition 2.2 (Chiffrement). *Pour chiffrer un bit b , on choisit un sous-ensemble aléatoire S d'indices parmi la clé publique et un bruit aléatoire r . Le texte chiffré c est calculé par :*

$$c = \left(b + 2r + \sum_{i \in S} x_i \right) \bmod x_0$$

Démonstration. Par définition du modulo x_0 , il existe $k \in \mathbb{Z}$ tel que :

$$c = b + 2r + \sum_{i \in S} x_i + kx_0$$

En remplaçant chaque x_i par son expression $q_i \cdot p + 2r_i$ (y compris pour x_0), on obtient :

$$c = b + 2r + \sum_{i \in S} (q_i \cdot p + 2r_i) + k(q_0 \cdot p + 2r_0)$$

En séparant les sommes et en regroupant par facteurs communs, on aboutit à :

$$c = p \left(\sum_{i \in S} q_i + kq_0 \right) + 2 \left(r + \sum_{i \in S} r_i + kr_0 \right) + b$$

On retrouve ainsi la structure d'un texte chiffré DGHV standard, ce qui justifie que $b = (c \bmod p) \bmod 2$. $\square$

2.4 Propriétés homomorphes et dynamique du bruit

Proposition 2.3 (Évaluation). *Soient $c_1 = pq_1 + 2r_1 + b_1$ et $c_2 = pq_2 + 2r_2 + b_2$ deux textes chiffrés.*

— $c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (b_1 + b_2)$

— $c_1 \cdot c_2 = p(q_1q_2p + \dots) + 2(2r_1r_2 + r_1b_2 + r_2b_1) + b_1b_2$

L'addition de textes chiffrés correspond à une porte XOR sur les clairs, et la multiplication à une porte AND.

Définition 2.3 (Budget de bruit et Seuil critique). Chaque schéma de chiffrement homomorphe possède un budget de bruit. Le déchiffrement n'est correct que si la valeur absolue du bruit accumulé respecte le seuil critique :

$$|2r + b| < \frac{p}{2}$$

Si ce seuil est franchi, l'opération modulo p (qui mathématiquement est un modulo centré ramenant les valeurs dans l'intervalle $]-p/2, p/2]$) va altérer la parité du résultat, rendant la récupération du bit impossible.

Exemple 2.2 (Déchiffrement correct vs Échec). Soit une clé secrète $p = 101$ (le seuil critique est donc 50.5).

- **Déchiffrement correct** : Chiffrons un bit $b = 1$ avec un bruit $2r = 10$. Le chiffré est $c = (2000 \times 101) + 10 + 1 = 202011$. On applique le modulo p : $202011 \bmod 101 = 11$. Puis $11 \bmod 2 = 1$. Le bit est correctement retrouvé car $11 < 50.5$.
- **Échec du déchiffrement** : Chiffrons un bit $b = 0$ et supposons qu'après plusieurs évaluations, le bruit ait explosé à $2r = 64$. Le chiffré est $c = (2000 \times 101) + 64 + 0 = 202064$. Lors du déchiffrement, l'opération modulo centré donne $[202064]_{101} = 64 \equiv -37 \pmod{101}$ car $64 > 50.5$. Ensuite, $-37 \bmod 2 = 1 \neq 0$. Le bit est corrompu, le déchiffrement a échoué.

Proposition 2.4 (Dynamique de la croissance du bruit). *L'évaluation homomorphe consomme le budget de bruit, mais de manière très asymétrique :*

- **Croissance linéaire (Addition)** : Le bruit du chiffré somme est $r_{add} = r_1 + r_2$. Cette opération est peu coûteuse.
- **Croissance explosive (Multiplication)** : Le produit $c_1 \cdot c_2$ génère un terme de bruit $R = 2(2r_1r_2 + r_1b_2 + r_2b_1) + b_1b_2$. Le nouveau bruit dominant est proportionnel à r_1r_2 , ce qui double approximativement la taille du bruit en bits à chaque multiplication.

Définition 2.4 (Profondeur multiplicative). La profondeur multiplicative désigne le nombre maximal de multiplications successives qu'un circuit peut appliquer à un chiffré avant que son budget de bruit ne dépasse le seuil critique $p/2$. Cette limite distingue le chiffrement *Somewhat Homomorphic* du chiffrement *Fully Homomorphic*.

2.5 Le Bootstrapping

Pour devenir "Fully Homomorphic", on utilise le rafraîchissement de texte chiffré. Le circuit de déchiffrement standard ($c \bmod p$) étant trop lourd mathématiquement, le DGHV utilise une clé publique augmentée d'un vecteur $\vec{y}$ de réels de haute précision. Le déchiffrement devient :

$$b = (c^* - \lfloor \sum s_i z_i \rfloor) \bmod 2$$

Cette opération génère suffisamment peu de bruit intrinsèque pour rendre l'évaluation homomorphe du déchiffrement (bootstrapping) possible, réinitialisant ainsi le budget de bruit d'un chiffré.

Chapitre 3

TFHE : Chiffrement sur le Tore

Chapitre 4

CKKS : Calcul sur les nombres réels

Bibliographie

- [1] Jung Hee CHEON et al. « Homomorphic Encryption for Arithmetic of Approximate Numbers ». In : *Advances in Cryptology – ASIACRYPT 2017*. Springer, 2017, p. 409-437.
- [2] Ilaria CHILLOTTI et al. « TFHE : Fast Fully Homomorphic Encryption Over the Torus ». In : *Journal of Cryptology* 33 (2020), p. 34-91.
- [3] Marten van DIJK et al. « Fully Homomorphic Encryption over the Integers ». In : *Advances in Cryptology – EUROCRYPT 2010*. Springer, 2010, p. 24-43.
- [4] FHE.ORG COMMUNITY. *History of FHE : A Timeline*. URL : <https://fhe.org/>.
- [5] Craig GENTRY. « A fully homomorphic encryption scheme ». Thèse de doct. Stanford University, 2009.
- [6] Pascal PAILLIER. *Introduction to Homomorphic Encryption*. FHE.org Meetup Presentation. 2020.
- [7] João Vítor Oliveira PALIARES et João MARTINELLI. *Explaining the DGHV Encryption Scheme*. 2024. URL : <https://medium.com/@j248360/explaining-the-dghv-encryption-scheme-1acb6cd74dd6>.
- [8] David J. WU. *Fully Homomorphic Encryption : Cryptography’s Holy Grail*. Rapp. tech. UT Austin Computer Science, 2015.